CS683: Advanced Computer Architecture, Autumn 2026

ACA Assignment 1

Task 2: Matrix multiplication for inference

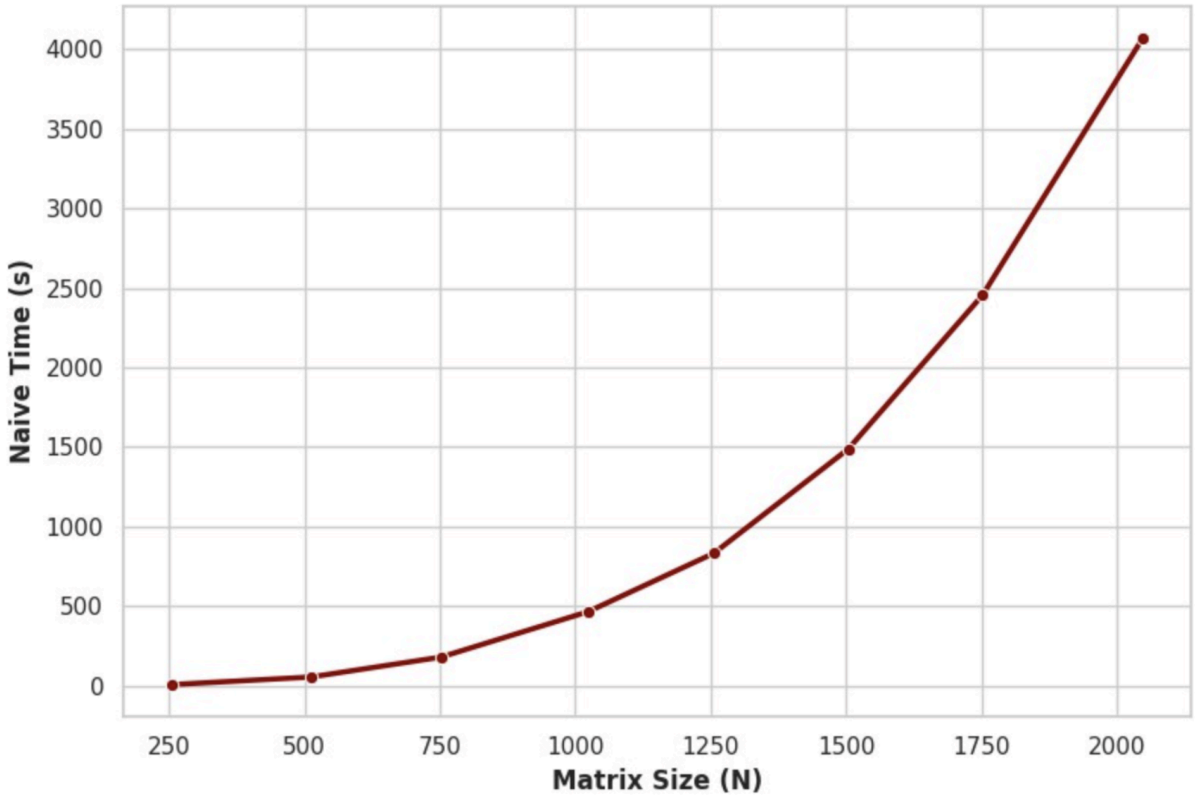
Naive Approach

The baseline matrix multiplication evaluates dot products across rows and columns using three nested loops in a straightforward, sequential manner.

Time Complexity: M x N x K

This implementation executes matrix multiplication utilizing a pre-transposed matrix B, which fundamentally optimizes the data memory access pattern. By structuring the arrays this way, the innermost loop traverses both matrix A and matrix B contiguously.

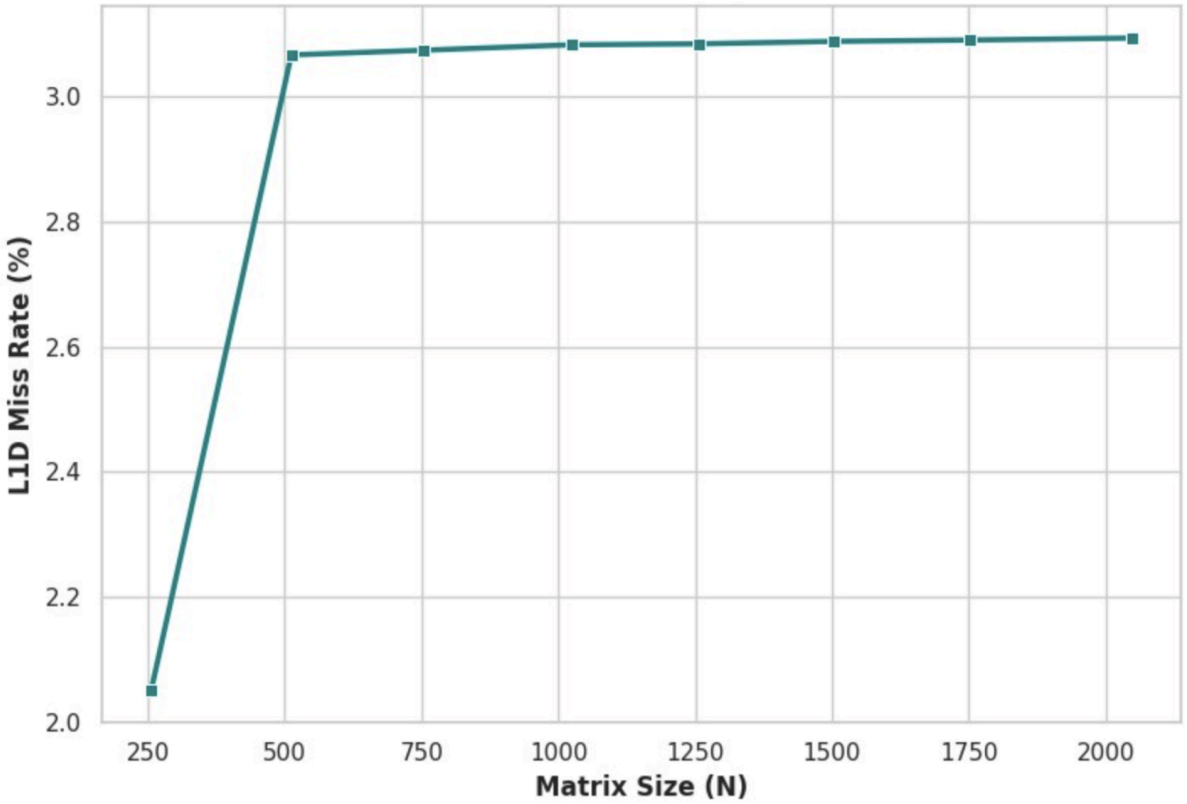
Execution Time VS Matrix Size:



The plot clearly illustrates the **O(N^3)** time complexity of the naive matrix multiplication algorithm.

- The steep polynomial curve demonstrates how rapidly execution time degrades as the problem size scales. This cubic growth is distinctly verifiable in the data points: when the matrix dimension doubles from N=1000 to N=2000, the execution time increases by a factor of approximately eight (jumping from roughly 500 seconds to over 4,000 seconds). This visualizes the severe computational bottleneck inherent to the unoptimized, triple-nested loop architecture before applying cache blocking or vectorization.

L1D Missrate Vs Matrix Size:



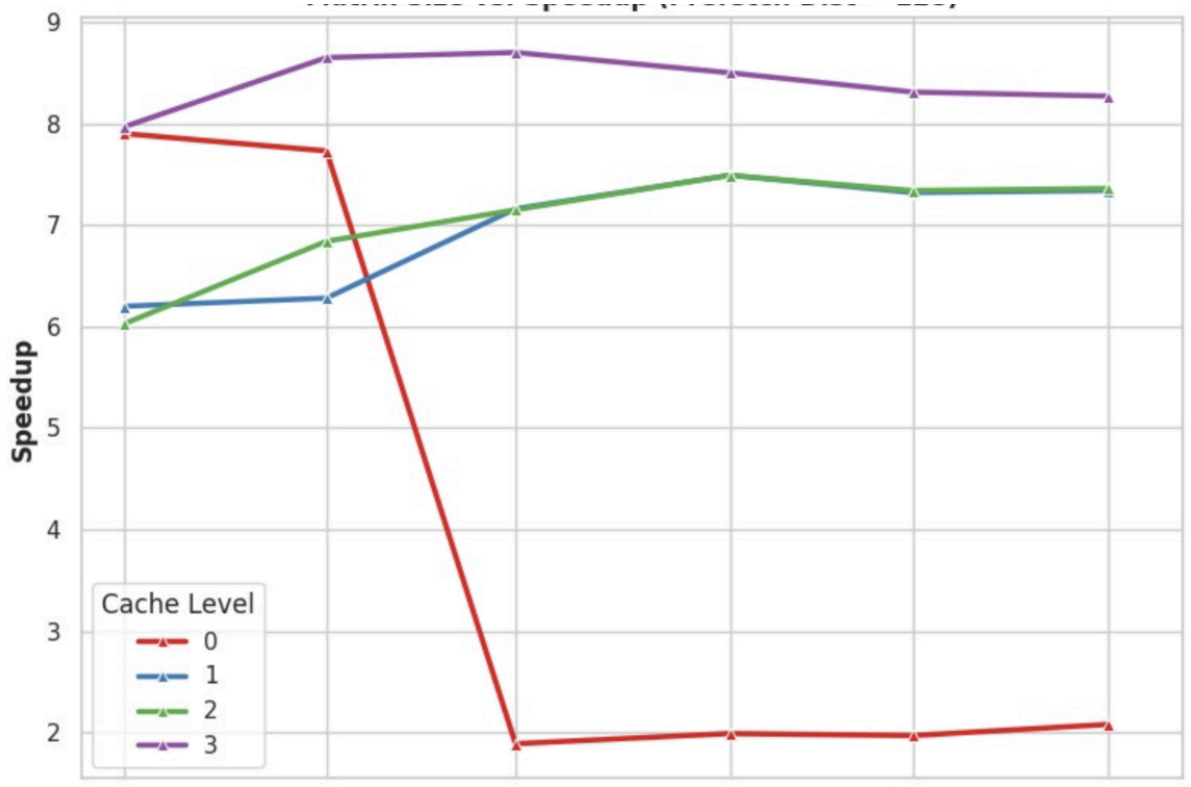
The plot illustrates the L1 Data (L1D) cache miss rate behavior for the unoptimized algorithm, characterized by two distinct architectural phases:

- At the smallest dimension (N=250), a significant portion of the working set still fits within the cache hierarchy, keeping the miss rate relatively low. As the matrix grows to N=500, its memory footprint rapidly exceeds the L1 cache capacity, causing a steep spike in capacity misses.
- For all sizes where >= 500, the miss rate completely flattens out just above 3.1%. This plateau signifies that the naive, column-wise memory traversal has reached its worst-case thrashing equilibrium. The cache is continuously evicting and fetching data at maximum frequency, creating a stable memory wall bottleneck that cannot degrade any further regardless of how large the matrices become.

Task 2A: Software prefetching:

In our implementation, we utilized **__builtin_prefetch** to explicitly stage data from matrices A and B about p_dist elements ahead of my current execution pointer directly into the cache. By integrating this targeted lookahead within my 64x64 cache-blocking and unrolled **256-bit SIMD** loops, we ensure the required arrays are readily available for the vector multiply-add operations. This active memory management effectively eliminates processor stalls, allowing my computational pipeline to maintain maximum execution throughput without waiting on main memory latency.

SpeedUp vs Size Vs Cache Level:

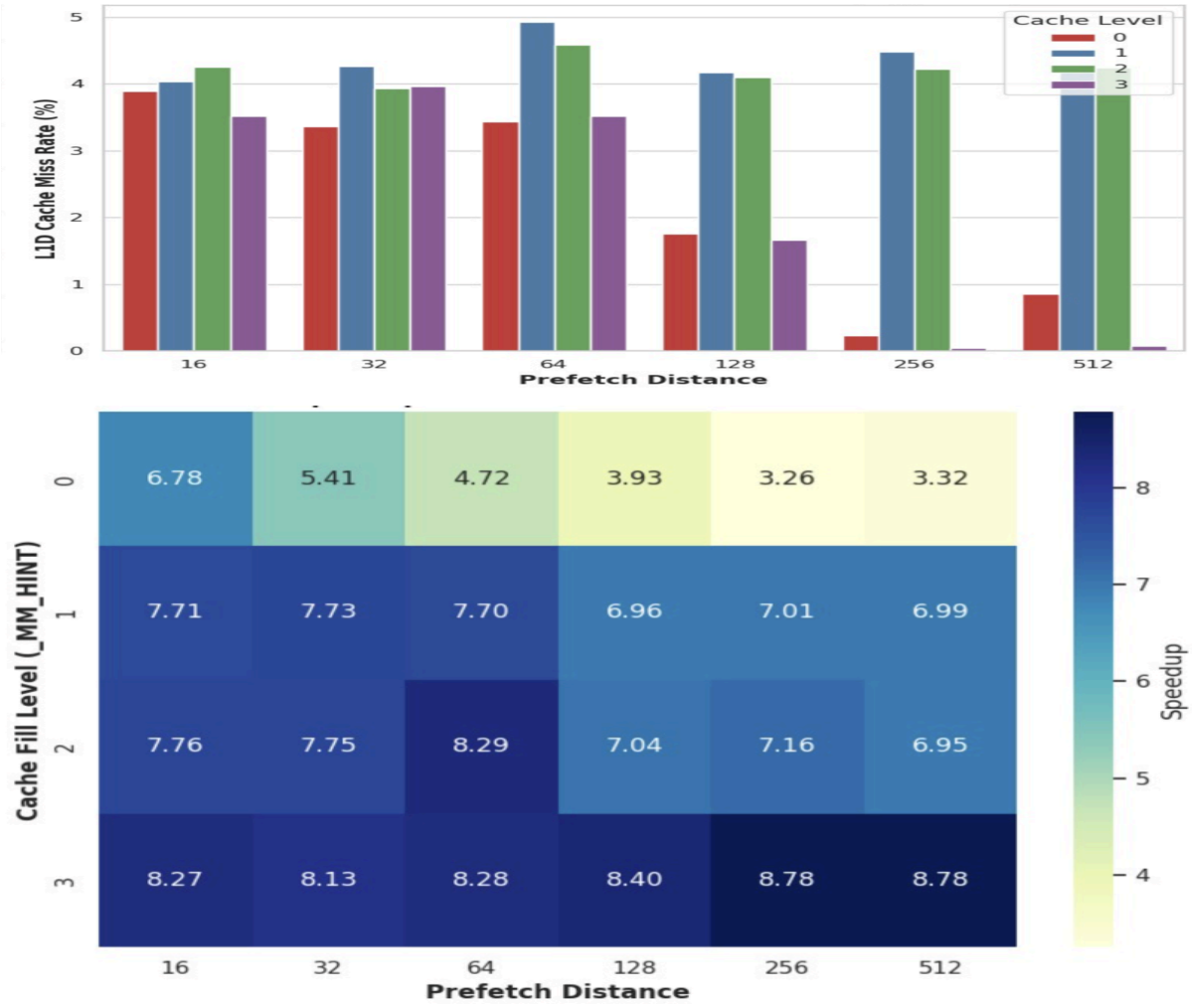


The plot illustrates performance scaling across different target cache levels at a fixed prefetch distance of 128:

- Targeting **Cache Level 3** (purple) consistently maintains the highest and most stable speedup across all matrix sizes. Its massive memory capacity safely buffers the 128-element lookahead without displacing the data required for immediate vector operations.

- Cache Level 0 (red) suffers a catastrophic performance collapse as matrix dimensions increase. While a prefetch distance of 128 is tolerable for very small matrices, scaling the problem size causes this aggressive lookahead to rapidly exceed the limited L1 capacity. This prematurely evicts the active working set, resulting in severe cache pollution and destructive thrashing.

Miss Rate vs Prefetch Distance vs Cache level



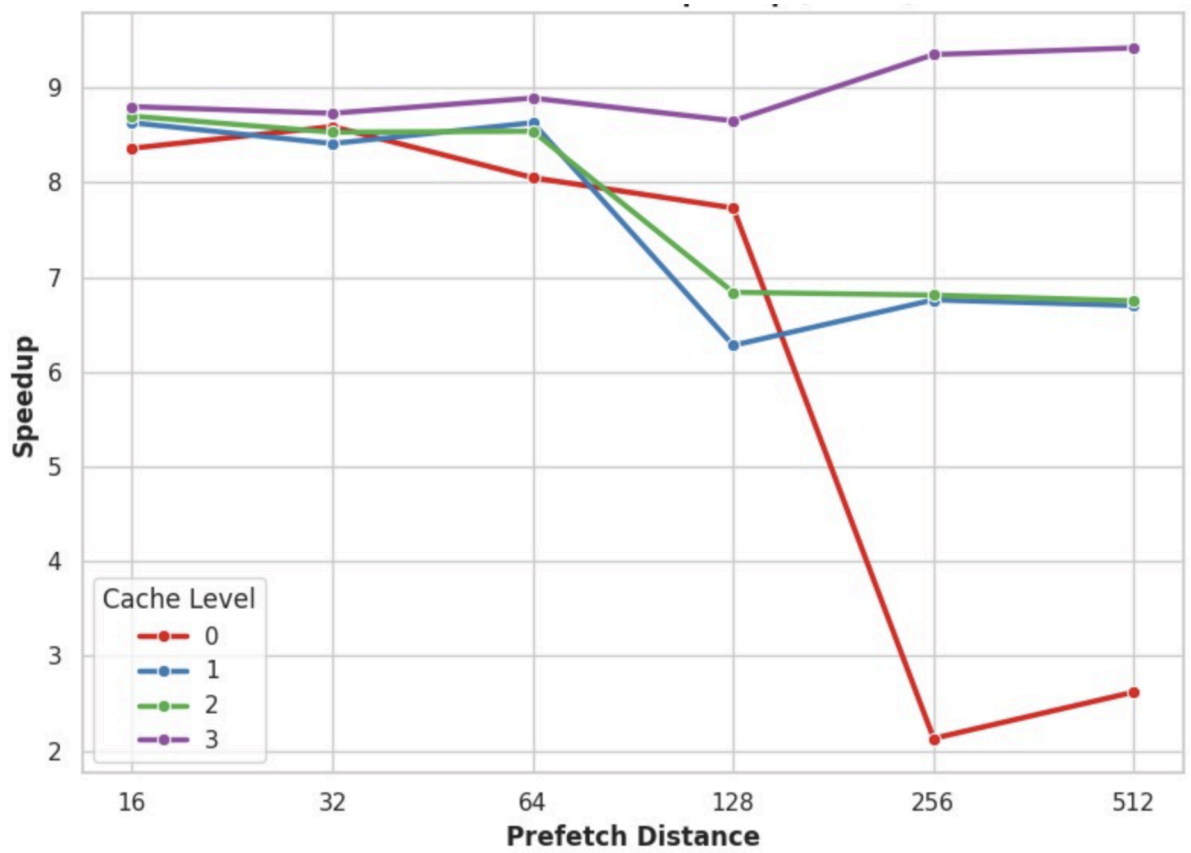
The provided visualizations illustrate the complex interplay between prefetch distances and target cache levels, revealing the optimal configuration for our memory subsystem:

- **Performance Heatmap:** This graph maps the parameter space, confirming that the optimal configuration is targeting the L3 cache (Level 3) with a large prefetch distance (256 or 512 elements), achieving a peak speedup

of 8.78x. Conversely, it starkly highlights the danger of cache pollution: prefetching too far ahead (256-512) directly into the small L1 cache (Level 0) halves the performance (down to ~3.26x) by prematurely evicting active data.

- **Cache Miss Rate:** This bar chart explains the *why* behind the heatmap's performance. When aggressively prefetching into Level 3 (purple bars at distances 256 and 512), the L1 Data (L1D) cache miss rate drops to near zero. Because the massive L3 capacity safely buffers the future data, it allows the hardware prefetcher to seamlessly stream elements into L1 exactly when the execution pipeline demands them, effectively eliminating memory stalls without destructive thrashing.

Speedup vs Dastance(Matrix Size = 512*512*512):



The plot for matrix size 512 demonstrates the critical balance between prefetch lookahead distance and hardware cache capacity:

- Target Cache Level 3 (purple) achieves its peak speedup (>9x) at large prefetch distances (256–512). Its massive capacity allows it to safely buffer data fetched far in advance without displacing the data needed for immediate computations.
- argeting smaller caches (Levels 0, 1, and 2) with large prefetch distances (>64) triggers severe performance degradation. Proactively fetching data too early into these restricted capacities prematurely evicts the active working set, leading to destructive cache thrashing. This is most distinctly illustrated by the drastic performance collapse of Level 0 at distance 256.

1. What trend do you observe in speedup with different matrix sizes?
    - As the matrix size increases, the **maximum attainable speedup generally decreases**. For example, the smallest matrix (size 256) achieves a peak speedup of 10.91x, whereas the largest matrix (size 1504) peaks at 8.27x. Additionally, the data shows that as matrix dimensions grow, targeting lower cache levels (Level 0) with large prefetch distances leads to severe performance degradation (dropping as low as 1.8x), indicating a transition to a heavily memory-bound state where capacity and thrashing become primary bottlenecks.
2. What is the best prefetch distance?
    - For sustained performance across diverse matrix sizes, a prefetch distance of **256 or 512 elements** is optimal. While shorter distances (e.g., 16) work well for very small matrices, larger lookaheads of 256 and 512 consistently deliver the highest and most stable speedups (typically ranging from 8.1x to 9.4x) for all matrix sizes > 256, provided the data is fetched into a sufficiently large cache.

3. At what cache fill level do you achieve the maximum speedup?

    - **Cache Level 3** provides the maximum stable speedup. While Cache Level 0 technically registers a single anomalous peak for the absolute smallest matrix size (10.91x at size 256), it suffers catastrophic capacity exhaustion and performance collapse on larger matrices. Cache Level 3 is the only configuration with the capacity to safely buffer large prefetch distances (256/512) across all matrix sizes without premature data eviction, maintaining consistent peak speedups above 8x.

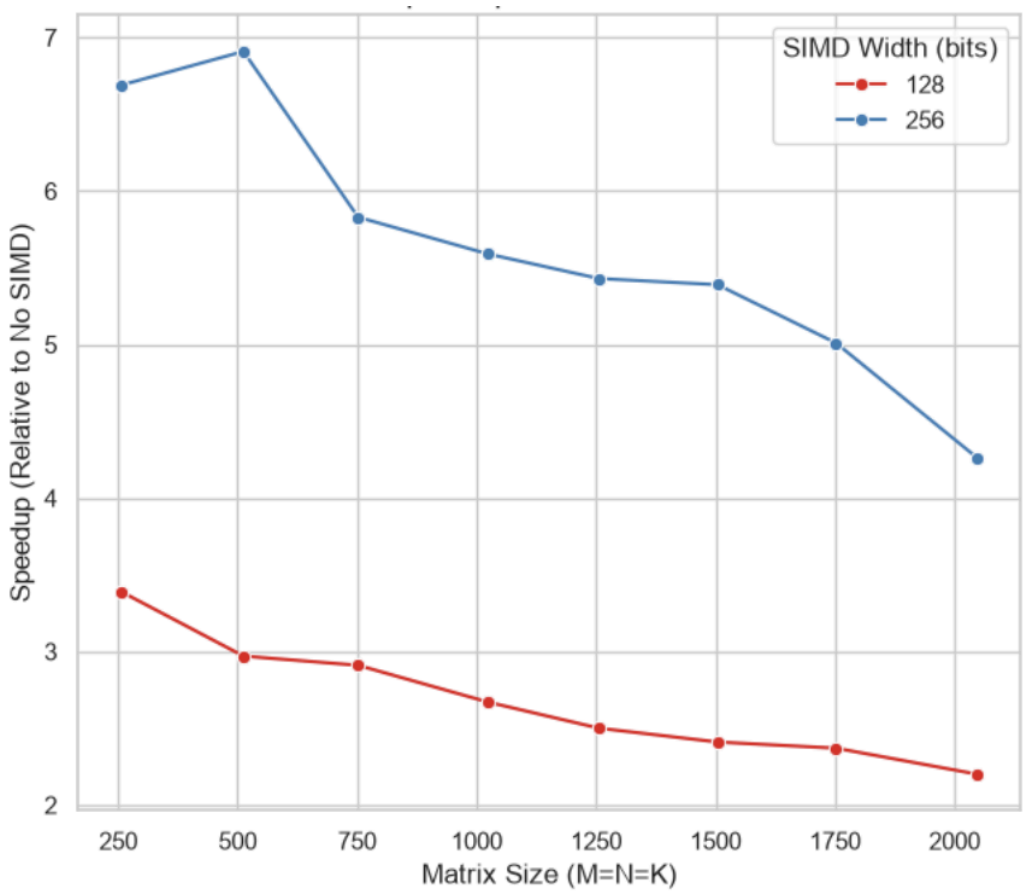
Task 2B: SIMD (Single Instruction, Multiple Data):

In our implementation, we utilized both **128-bit** and **256-bit** AVX/AVX2 intrinsics to process multiple single-precision floating-point elements simultaneously per loop iteration. By leveraging aligned vector loads and hardware-level fused multiply-add operations (such as **_mm256_fmadd_ps**), we vectorized the innermost dot-product calculations to drastically reduce total instruction counts and benchmark the scaling performance across different SIMD widths. We also incorporated a scalar fallback loop to handle any trailing matrix dimensions that do not perfectly align with our chosen vector lengths, ensuring mathematical correctness while maximizing execution throughput.

Matrix operations performance:

Metric Type		No SIMD		SIMD	Speedup (normalized to no SIMD)
Metrics	Instructions	Execution time (s)	Instructions	Execution time (s)	
Size 256 | 128-bit	835882860.000	7.519	811070136.000	1.725	3.390
Size 256 | 256-bit	619834302.000	6.206	688895737.000	0.884	6.690
Size 512 | 128-bit	4887251955.000	52.017	5761877590.000	17.571	2.970
Size 512 | 256-bit	4887340897.000	52.689	5373154310.000	7.671	6.910
Size 752 | 128-bit	15636796335.000	170.123	18165568540.000	58.473	2.910
Size 752 | 256-bit	15621104836.000	170.040	16946052184.000	29.626	5.830
Size 1024 | 128-bit	38878949541.000	443.529	45882780116.000	165.124	2.670
Size 1024 | 256-bit	38879845396.000	440.438	42501127768.000	79.536	5.590
Size 1256 | 128-bit	71678430744.000	843.908	84271844718.000	324.269	2.500
Size 1256 | 256-bit	71670793892.000	811.359	78355598875.000	148.773	5.430
Size 1504 | 128-bit	122974660590.000	1423.498	144549522479.000	589.837	2.410
Size 1504 | 256-bit	122972289485.000	1417.374	134183048273.000	268.308	5.390
Size 1752 | 128-bit	194298714323.000	2369.483	228347449954.000	1002.460	2.370
Size 1752 | 256-bit	194297600358.000	2367.095	211884621946.000	462.720	5.010
Size 2048 | 128-bit	310214653225.000	3875.733	364478480703.000	1754.006	2.200
Size 2048 | 256-bit	310201562078.000	3784.166	338161593139.000	926.194	4.260

Speedup Vs Matrix Size:



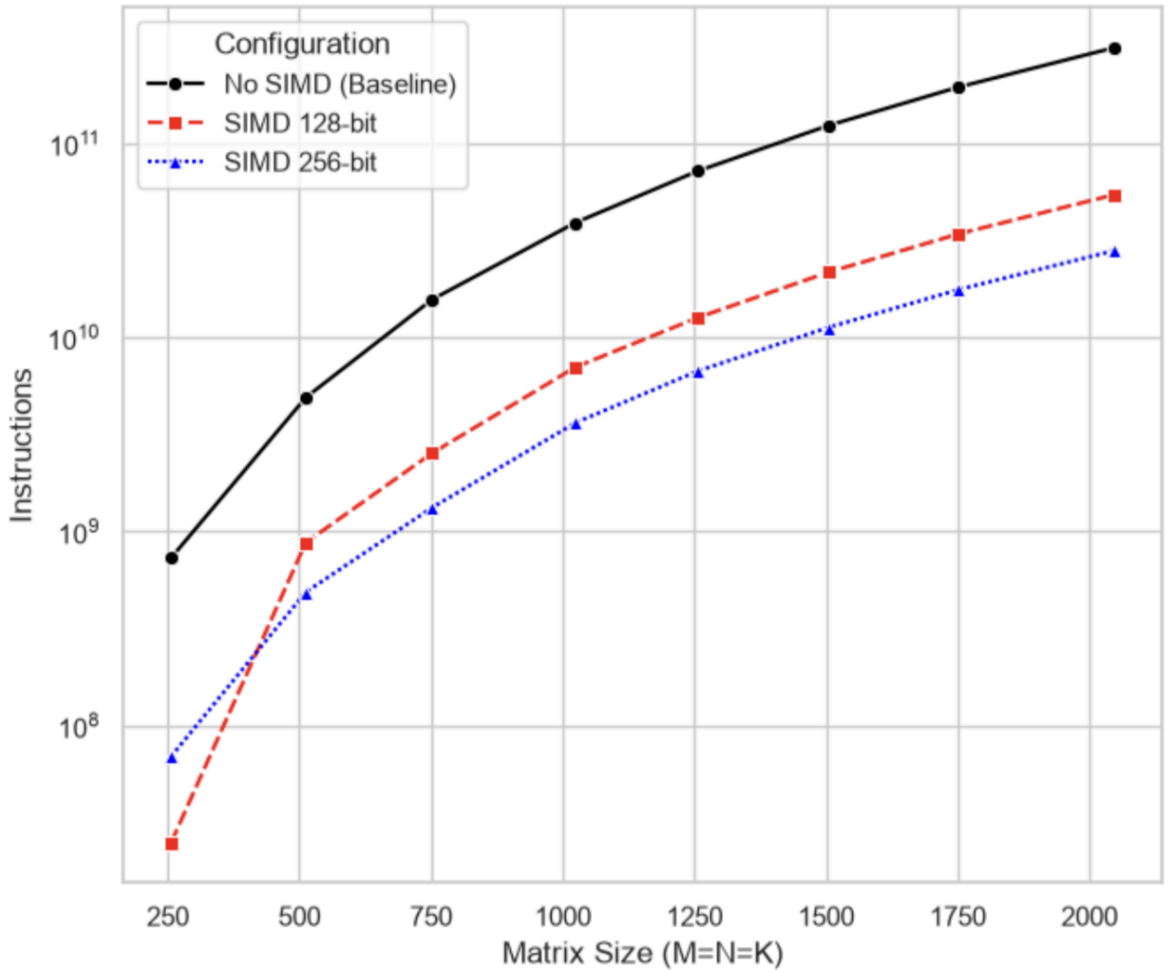
The plot illustrates the performance speedup of 128-bit and 256-bit SIMD implementations relative to a non-SIMD baseline across varying matrix sizes. The data demonstrates two key architectural behaviors:

- The 256-bit implementation consistently outperforms the 128-bit configuration, initially delivering roughly double the speedup (peaking near 7x compared to the 128-bit's 3.5x). This aligns with theoretical expectations, as the wider registers process twice as many floating-point elements per instruction.

- Both configurations exhibit a steady decline in relative speedup as the matrix dimensions increase. For smaller matrices (e.g., sizes 250–500), the data fits entirely within the high-speed CPU caches, allowing the SIMD units to

operate at peak efficiency. As the matrices grow larger, they exceed cache capacities, and the system becomes increasingly bottlenecked by main memory bandwidth and fetch latency, preventing the execution pipeline from maintaining maximum throughput.

Instructions VS Matrix Size:



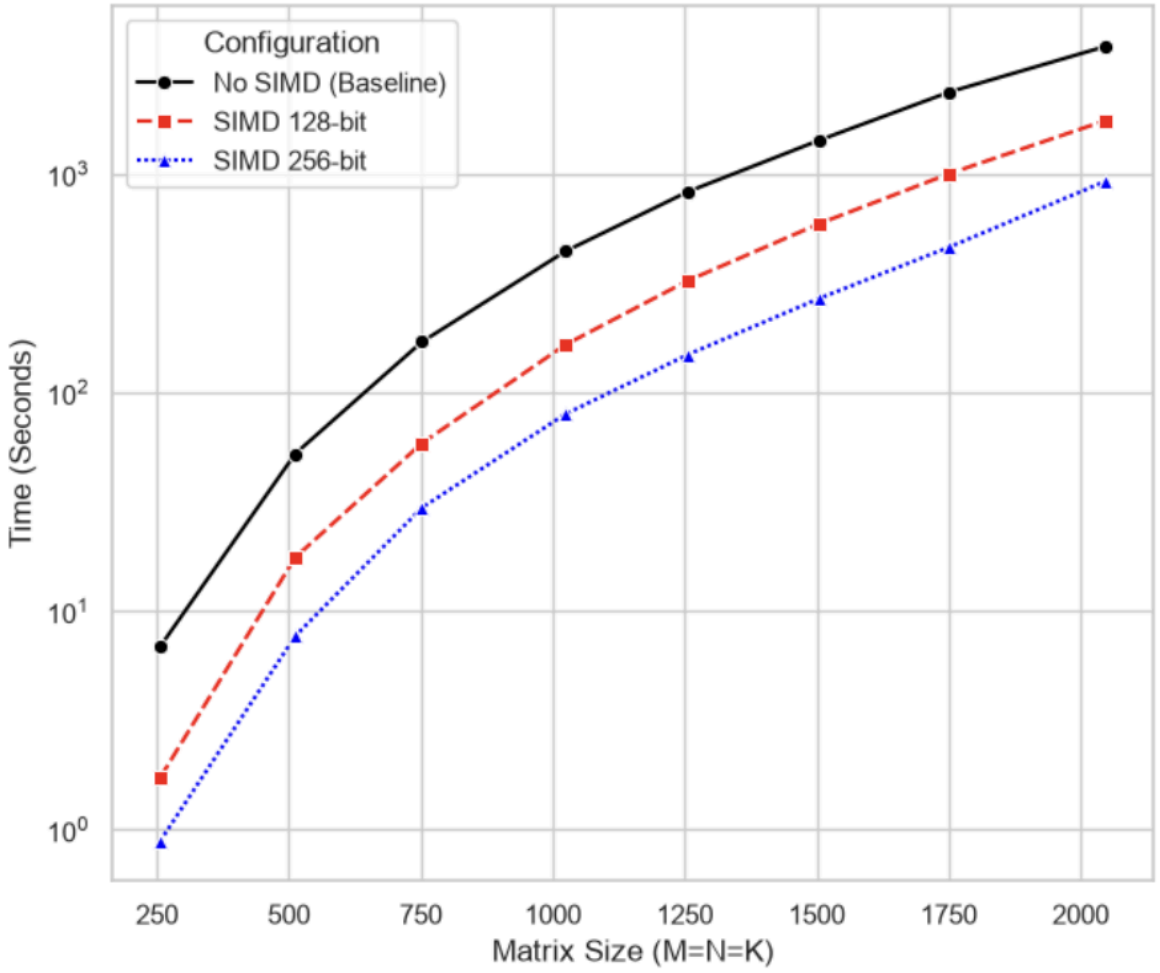
The plot highlights two critical architectural behaviors regarding dynamic instruction counts:

- All three configurations track along similar upward trajectories across the logarithmic y-axis. This confirms that while vectorization processes more data per instruction, it does not alter the fundamental O(N^3)

computational complexity of the matrix multiplication algorithm.

- The graph clearly demonstrates that SIMD *drastically* reduces the total number of instructions executed. By processing 4 or 8 floating-point numbers in a single operation, the SIMD 128-bit and SIMD 256-bit configurations successfully eliminate billions of core loop iterations and scalar instructions compared to the No SIMD baseline.

Execution Time VS Matrix Size:



The plot demonstrates two key behaviors:

- The 256-bit SIMD configuration consistently delivers the lowest execution times, proving that wider vector

registers successfully translate hardware parallelism into faster wall-clock performance.

- All three curves exhibit identical growth trajectories on the logarithmic scale, confirming that while vectorization improves constant-factor speed, it does not alter the fundamental O(N^3) computational scaling.

Questions:

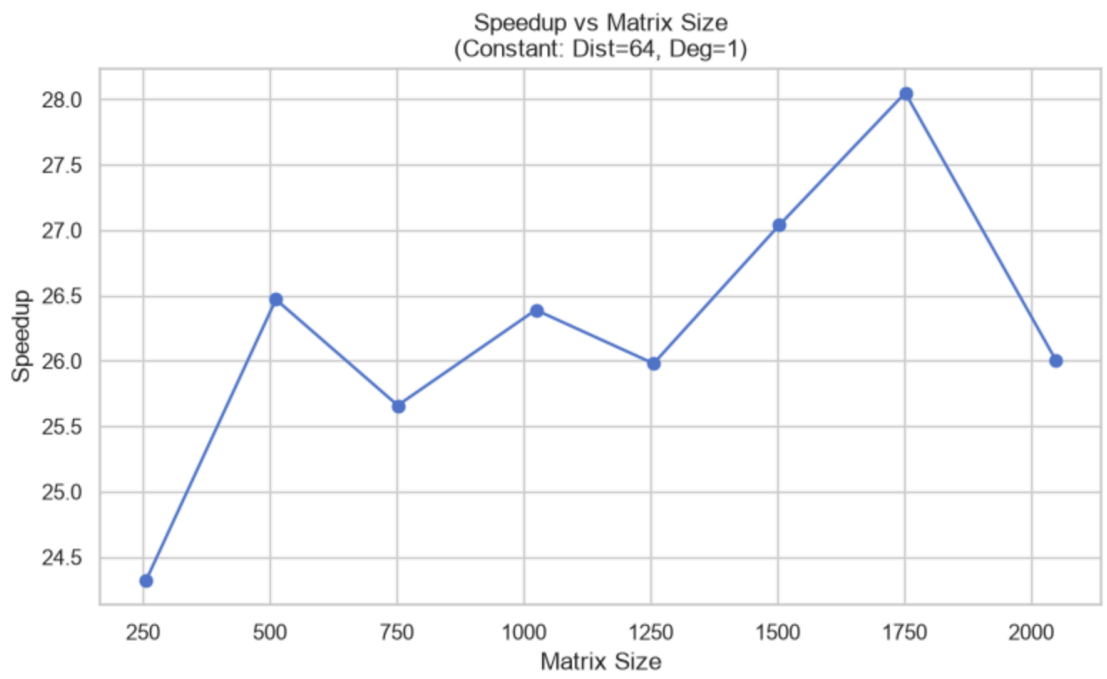
1. What trends do you observe in speedup for different combinations of matrix sizes and SIMD widths?
    - The 256-bit SIMD width consistently achieves a significantly higher speedup compared to the 128-bit width across all tested matrix sizes.
    - For both SIMD configurations, the relative speedup peaks at the smallest matrix sizes (where data easily fits within CPU caches) and steadily declines as matrix sizes increase (where the workload becomes bottlenecked by main memory bandwidth and fetch latency).
2. For which SIMD width do you achieve the maximum speedup?
    - The maximum speedup is achieved using the **256-bit SIMD width** on the **smallest matrix size** tested (M=N=K=256), peaking at approximately **7x** relative to the non-SIMD baseline.

Task 2C: Optimized Solution

For the optimized matrix multiplication the approaches we took are:

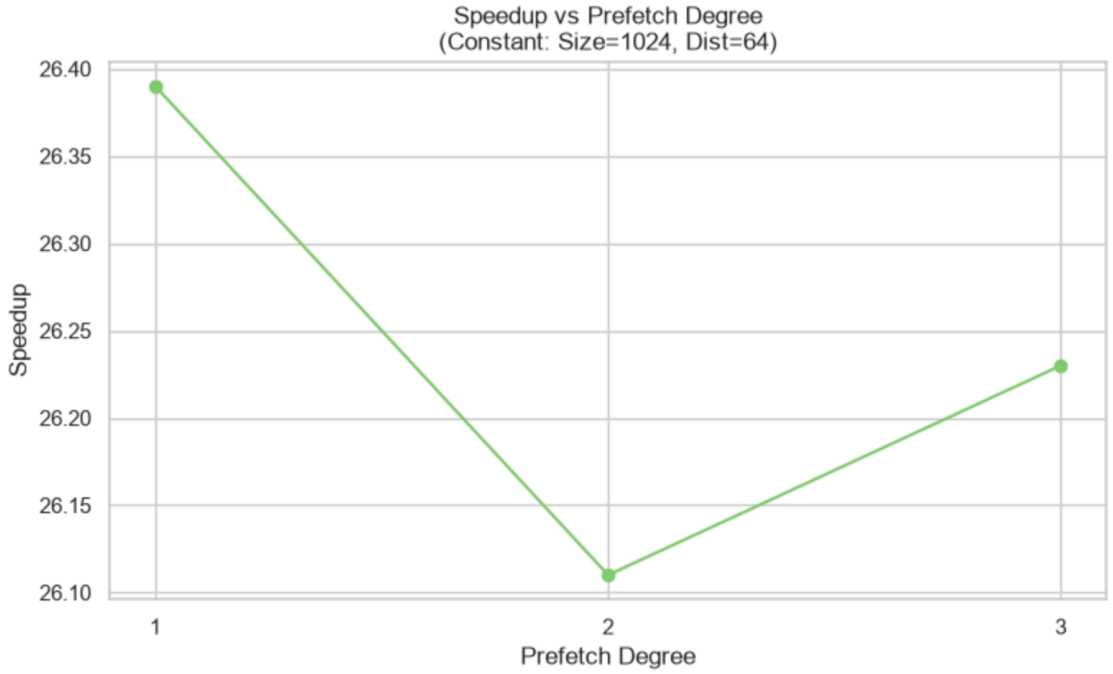
- **Two-Level Memory Blocking:** To eliminate the memory wall, we implemented a dual-layered blocking strategy. The outer loops divide the matrices into **96x64 cache blocks** to ensure the working set stays resident in the fast L1/L2 caches, preventing memory thrashing. Inside these blocks, we utilized a **3x4 register tiling** technique-computing 12 distinct elements of the output matrix C simultaneously. This maximizes instruction level parallelism and heavily reuses the A and B matrix values already loaded into the CPU registers.
- **AVX2 SIMD Vectorization:** We fully vectorized the innermost computational core using 256-bit AVX2 intrinsics. By calculating eight floating-point elements per cycle using hardware-level fused multiply-add (_mm256_fmadd_ps) instructions across our 3x4 register grid, we drastically reduced the total dynamic instruction count and pushed the hardware closer to its theoretical peak computational throughput.
- **Software Prefetching & Edge Handling:** To actively hide main memory latency, we injected __builtin_prefetch instructions to stage future elements of matrix B 128 indices ahead of the execution pointer directly into the cache hierarchy. Finally, we engineered robust fallback loops at the end of the execution block to independently process any matrix boundaries that do not perfectly align with our 3x4 tiles or 8-element vector widths, guaranteeing absolute mathematical correctness across all matrix sizes.

Speedup vs. Matrix Size (Dist=64, Deg=1)

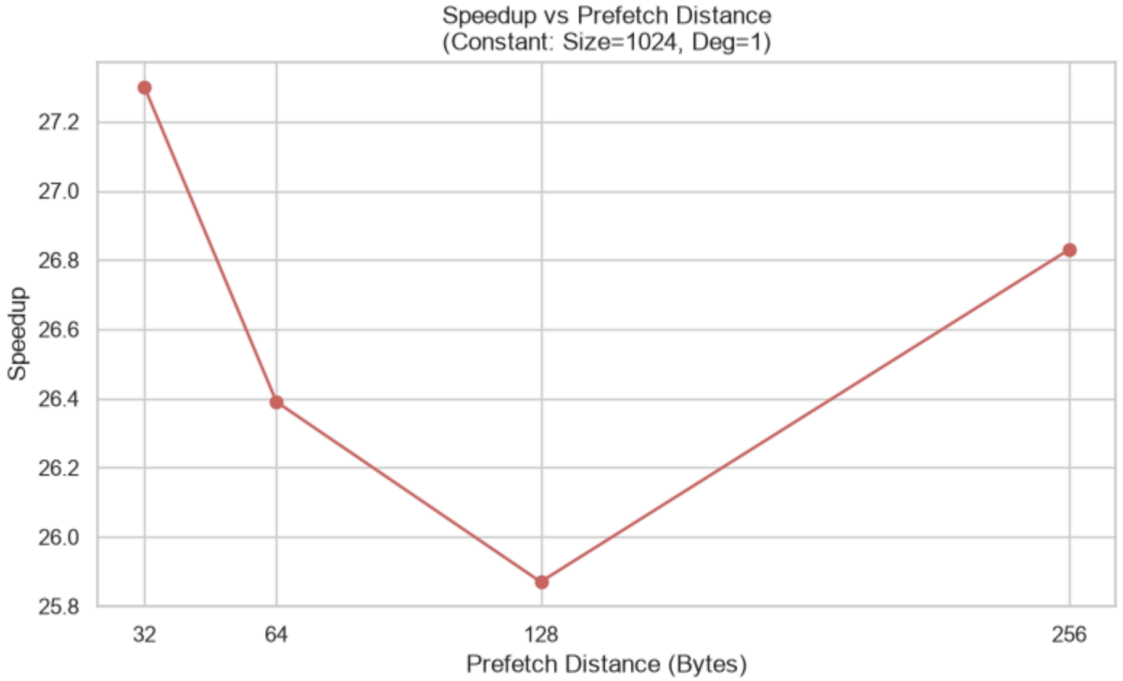


- The kernel delivers a strong 24x to 28x speedup across all tested dimensions, confirming that the 96x64 cache blocking and AVX intrinsics effectively overcome memory bandwidth limits.
- The visible performance dips (such as at N=752 and N=1256) occur when matrix sizes do not divide evenly by the block dimensions, causing the kernel to fall back on slower scalar loops to process the leftover matrix edges.

Speedup vs. Prefetch Degree (Size=1024, Dist=64)



- A prefetch degree of 1 yields the highest speedup (~26.39x), while increasing the degree to 2 or 3 introduces a slight performance regression.
- Fetching multiple cache lines per loop iteration likely overwhelms the memory subsystem or L1 cache, prematurely evicting useful active data and adding unnecessary instruction overhead without providing additional latency-hiding benefits.



- Performance peaks at a 32-byte prefetch distance (~27.3x speedup), experiences a noticeable drop at 128 bytes, and partially recovers at 256 bytes.
- The 32-byte distance perfectly synchronizes with the AVX256 vector loads (__m256), ensuring data is fetched exactly when needed. Larger distances disrupt this timing, causing data to arrive too early and risk eviction before the heavily unrolled micro-kernel can process it.